

VYBE — Lifestyle & Tech Store

Technical Report: Implemented Features and Component Architecture

Prepared for: Project Review

Subject: Vue 3 / TypeScript Single-Page Application

Date: 22 September 2026

1. Executive Summary

VYBE is a client-side, data-driven e-commerce single-page application (SPA) built with Vue 3, TypeScript, and Tailwind CSS. The application retrieves product data from the public DummyJSON REST API and simulates a full retail shopping experience, including browsing, filtering, cart management, wishlists, authentication, and checkout. State is managed centrally with Pinia, and the majority of user-specific data is persisted to the browser's local storage so that sessions survive a page reload. This report documents the features implemented in the application and describes the hierarchy and responsibilities of its components.

2. Features Implemented

2.1 Product Discovery

- **Product browsing** — a paginated/scrollable product catalogue retrieved from the DummyJSON API and rendered through a reusable grid component.
- **Product detail view** — a dedicated route (`/product/:id`) presenting extended information for a single product.
- **Category filtering** — a dedicated category route (`/category/:slug`) that filters the catalogue by category slug.
- **Search and filtering** — client-side search and filter controls layered over the product listing.

2.2 Account & Session Management

- **Authentication simulation** — login and signup flows backed by a simulated authentication service, including credential validation against locally stored signup records.
- **Session persistence with expiry** — an authentication token, refresh token, and user profile are persisted to local storage together with a 30-minute expiry timestamp, allowing the session to be validated or invalidated on reload.
- **Route redirect handling** — unauthenticated users attempting to access protected actions are redirected to the login page, with the original destination preserved for post-login redirect.
- **Profile page** — displays the current authenticated user's account details.

2.3 Shopping Experience

- **Shopping cart** — add, remove, and update item quantities, with derived item count and total price getters; cart contents persist to local storage between sessions.
- **Wishlist / bookmarks** — a bookmark store allows products to be saved and toggled, with persisted state and a fast lookup for "is this product bookmarked" checks.
- **Checkout flow** — a dedicated checkout page followed by an order confirmation (order success) page.
- **Order history** — a utility module records completed orders for later reference.

2.4 User Interface & Experience

- **Dark mode** — a toggleable theme that respects the operating system's preferred colour scheme by default and persists the user's explicit choice.
- **Toast notifications** — a global, auto-dismissing notification system for confirming actions such as "added to cart".
- **Country / region preference** — a persisted locale-style preference selectable from a full list of countries.
- **Responsive design** — layout and spacing scale across mobile, tablet, and desktop breakpoints using Tailwind CSS utility classes.
- **Global error handling** — page-level errors are captured centrally, presented with a descriptive message, and recoverable via a "Try Again" action that reloads the current route.
- **Not-found handling** — a catch-all route presents a dedicated 404 page for unmatched paths.

2.5 Feature Summary Table

Area	Feature	Primary Module(s)
Catalogue	Product browsing, search, filtering	ProductsPage, ProductGrid, productService
Catalogue	Product detail view	ProductDetailPage, productService
Account	Login / signup / session expiry	LoginPage, SignupPage, authStore, authService
Shopping	Cart with persisted totals	CartPage, cartStore
Shopping	Wishlist / bookmarks	WishlistPage, bookmarkStore
Shopping	Checkout & order confirmation	CheckoutPage, OrderSuccessPage
UX	Dark mode	useDarkMode composable
UX	Toast notifications	useToast composable, ToastStack
UX	Country preference	useCountryPreference composable
Reliability	Global error boundary	App.vue (onErrorCaptured)

3. Component Architecture

3.1 Architectural Overview

The application follows a layered, unidirectional architecture typical of a Vue 3 single-page application. Presentation concerns (pages and reusable components) are kept separate from cross-cutting UI behaviour (composables), application state (Pinia stores), and data access (services and a thin HTTP wrapper). Each layer depends only on the layer beneath it, which keeps components free of direct network logic and keeps state management independent of any single page.

3.2 Layer Descriptions

Layer	Responsibility
Application Shell	App.vue mounts the persistent layout, hosts the routed page outlet inside a Suspense boundary, and captures unhandled page errors.
Layout & Global UI	Navbar, Footer, and ToastStack are rendered on every route and provide navigation and global notifications.
Routed Page Views	One component per route (Home, Products, Product Detail, Login, Signup, Cart, Checkout, Order Success, Wishlist, Profile, Not Found), each responsible for a single screen's layout and data orchestration.
Reusable Feature Components	ProductGrid and ProductCard are shared presentation components used by both the Products page and the Wishlist page to render collections of products consistently.
Composables & State	Composables (useDarkMode, useToast, useCountryPreference) encapsulate reusable stateful behaviour; Pinia stores (authStore, cartStore, bookmarkStore) hold shared application state and persist it to local storage.
Services	authService and productService expose typed functions for authentication and catalogue operations, isolating stores and pages from raw HTTP calls.
HTTP Transport & External Data	api.ts provides a single apiFetch wrapper around the Fetch API and the DummyJSON REST API, which is the external system of record for product data.

3.3 Component Hierarchy Diagram

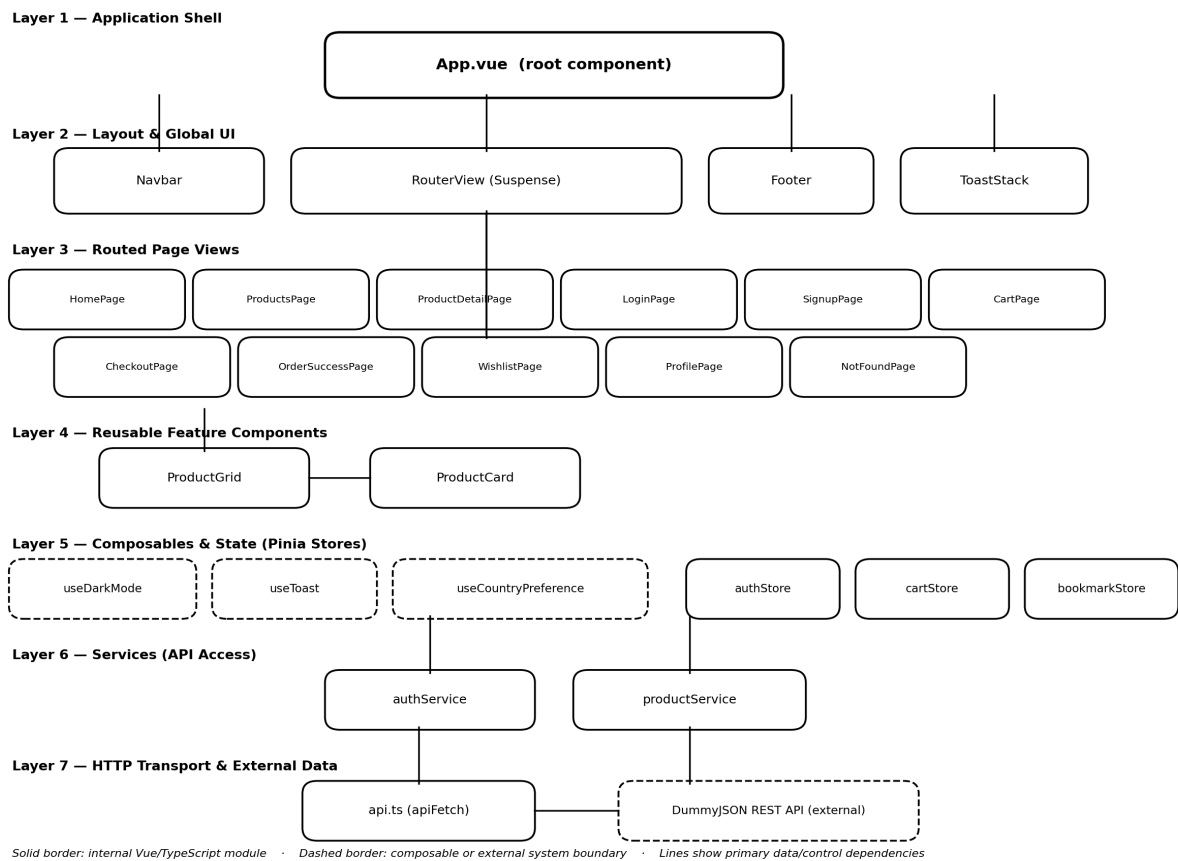


Figure 1. Component and module hierarchy, from the root application shell down to the external REST API. Solid-border boxes are internal Vue or TypeScript modules; dashed-border boxes are composables or the external API boundary. Vertical connectors indicate a parent-to-child or consumer-to-dependency relationship.

3.4 Component Tree (Text Form)

```
App.vue
|-- Navbar
|-- RouterView (Suspense-wrapped route outlet)
|   |-- HomePage
|   |-- ProductsPage --> ProductGrid --> ProductCard
|   |-- ProductDetailPage
|   |-- LoginPage
|   |-- SignupPage
|   |-- CartPage
|   |-- CheckoutPage
|   |-- OrderSuccessPage
|   |-- WishlistPage --> ProductGrid --> ProductCard
|   |-- ProfilePage
|   |-- NotFoundPage
|-- Footer
`-- ToastStack
```

3.5 State and Data Flow

Pages read and mutate state exclusively through Pinia stores rather than holding duplicate local copies of shared data. For example, CartPage dispatches actions on cartStore to add, remove, or update quantities, and reads the itemCount and totalPrice getters for display; LoginPage and SignupPage operate on authStore, which in turn calls authService; and WishlistPage reads and toggles entries in bookmarkStore. Both cartStore and bookmarkStore persist their item arrays to local storage on every mutation, so cart and wishlist contents survive a browser refresh. Product data itself is not stored in Pinia; pages call productService directly, which delegates to the shared apiFetch wrapper in api.ts to query the DummyJSON REST API.

3.6 Cross-Cutting Concerns

- **Composables** encapsulate reusable, non-visual behaviour that would otherwise be duplicated across components: useDarkMode manages the theme class on the document root, useToast manages a global notification queue, and useCountryPreference manages a persisted locale selection.
- **Error handling** is centralised in App.vue using Vue's onErrorCaptured hook, so any uncaught error thrown while rendering a routed page is intercepted once, displayed with a recovery action, and cleared automatically on navigation.
- **Persistence** is handled consistently through the browser's local storage across authentication, cart, bookmarks, and user preferences, avoiding the need for a backend session store.

4. Technology Stack

Concern	Technology
Framework	Vue 3 (Composition API, script setup)
Language	TypeScript
Build tool	Vite
Styling	Tailwind CSS 4
State management	Pinia
Routing	Vue Router
Data source	DummyJSON public REST API

5. Conclusion

VYBE implements the core functionality expected of a modern e-commerce SPA — product discovery, account management, cart and wishlist handling, and checkout — while maintaining a clean separation between presentation, state, and data-access layers. The layered component hierarchy allows each page to remain a thin orchestration layer, with shared behaviour consolidated into composables, stores, and services that can be reused, tested, and extended independently of the user interface.